COURSE NAME: OPERATIN SYSTEMS

COURSE CODE: BCS303

SEMESTER: 3

MODULE: 2

NUMBER OF HOURS:08

CONTENTS:

- **Process Management:**
  - Process concept
  - Process scheduling
  - Operations on processes
  - Inter process communication

- **Multi-threaded Programming:**
  - Multithreading models
  - Thread Libraries
  - Threading issues

- **Process Scheduling:**
  - Basic concepts
  - Scheduling Criteria;
  - Scheduling Algorithms;
  - Multiple-processor scheduling;
  - Thread scheduling.

PROCESS MANAGEMENT

Process Concept

- A process is a program under execution.
- Its current activity is indicated by PC (Program Counter) and the contents of the processor's registers.

The Process

Process memory is divided into four sections as shown in the figure below:

- The **stack** is used to store temporary data such as local variables, function parameters, function return values, return address etc.
- The **heap** which is memory that is dynamically allocated during process run time
- The **data** section stores global variables.
- The **text** section comprises the compiled program code.
- Note that, there is a free space between the stack and the heap. When the stack is full, it grows downwards and when the heap is full, it grows upwards.

Process State

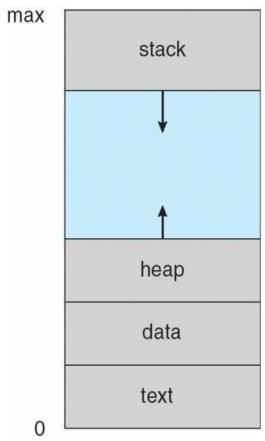


Figure: Process in memory.

Q) Illustrate with a neat sketch, the process states and process control block.

Process State

A Process has 5 states. Each process may be in one of the following states –

1. **New** - The process is in the stage of being created.
2. **Ready** - The process has all the resources it needs to run. It is waiting to be assigned to the processor.
3. **Running** – Instructions are being executed.
4. **Waiting** - The process is waiting for some event to occur. For example, the process may be waiting for keyboard input, disk access request, inter-process messages, a timer to go off, or a child process to finish.
5. **Terminated** - The process has completed its execution.

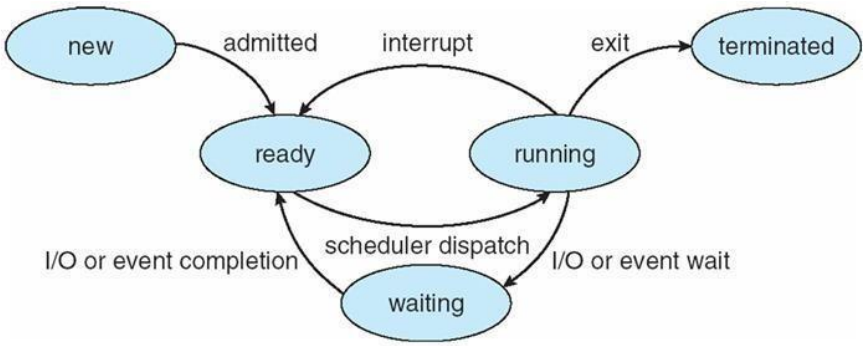


Figure: Diagram of process state

Process Control Block

For each process there is a Process Control Block (PCB), which stores the process-specific information as shown below –

- **Process State** – The state of the process may be new, ready, running, waiting, and so on.
- **Program counter** – The counter indicates the address of the next instruction to be executed for this process.
- **CPU registers** - The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general-purpose registers. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward.
- **CPU scheduling information**- This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.
- **Memory-management information** – This includes information such as the value of the base and limit registers, the page tables, or the segment tables.
- **Accounting information** – This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
- **I/O status information** – This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

The PCB simply serves as the repository for any information that may vary from process to process.

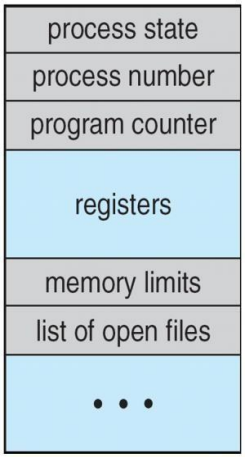


Figure: Process control block (PCB)

CPU Switch from Process to Process

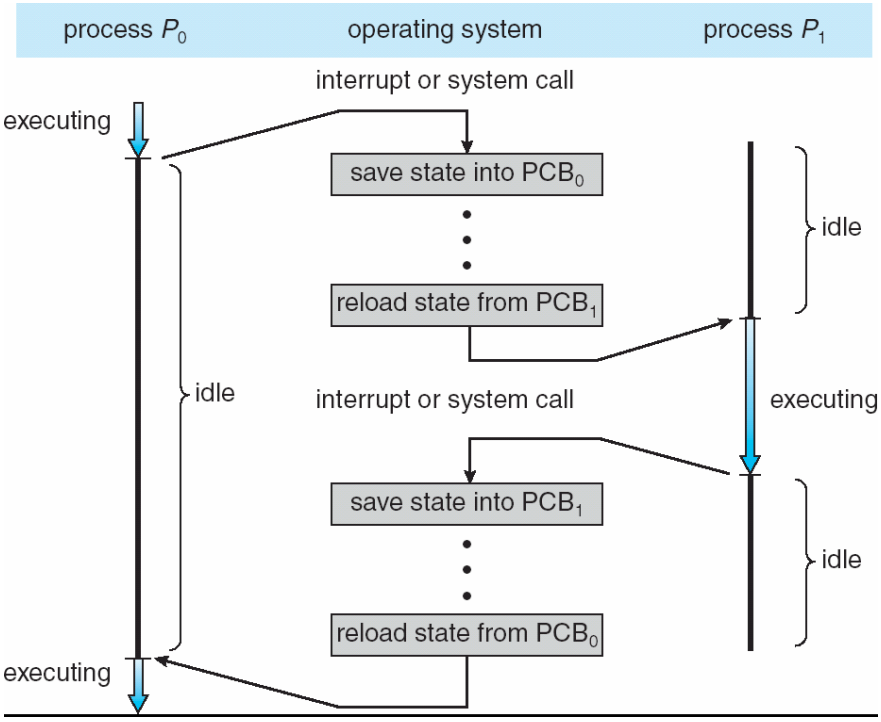


Figure: Diagram showing CPU switch from process to process.

Process Scheduling

Scheduling Queues

- As processes enter the system, they are put into a job queue, which consists of all processes in the system.
- The processes that are residing in main memory and are ready and waiting to execute are kept on a list called the ready queue. This queue is generally stored as a linked list.
- A ready-queue header contains pointers to the first and final PCBs in the list. Each PCB includes a pointer field that points to the next PCB in the ready queue.

Ready Queue and Various I/O Device Queues

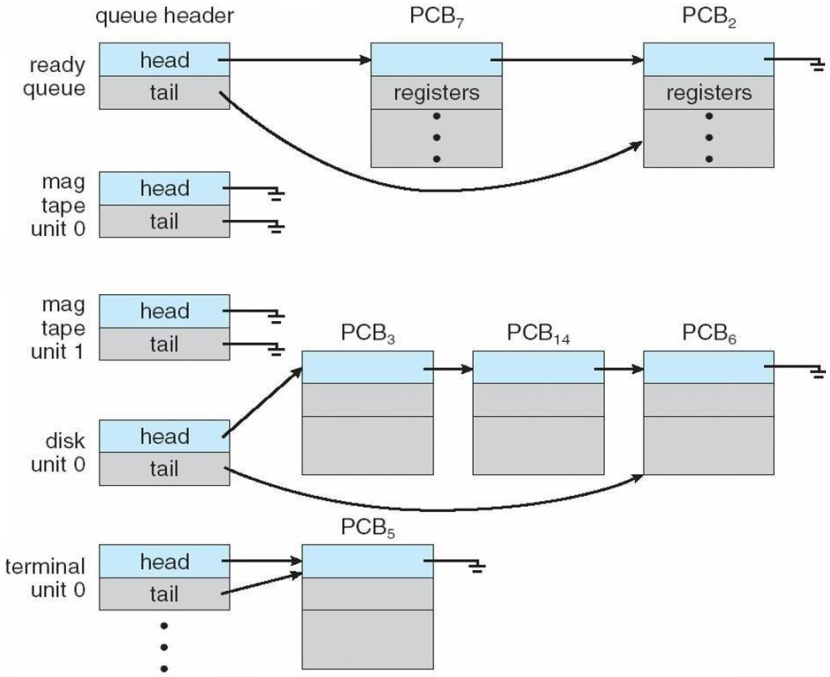


Figure: The ready queue and various I/O device queues

- A common representation of process scheduling is a ***queueing diagram***. Each rectangular box in the diagram represents a queue. Two types of queues are present: the ready queue and a set of device queues. The circles represent the resources that serve the queues, and the arrows indicate the flow of processes in the system.
- A new process is initially put in the ready queue. It waits in the ready queue until it is selected for execution and is given the CPU. Once the process is allocated the CPU and is executing, one of several events could occur:
    - The process could issue an I/O request, and then be placed in an I/O queue.
    - The process could create a new subprocess and wait for its termination.
    - The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.

In the first two cases, the process eventually switches from the waiting state to the ready state, and is then put back in the ready queue. A process continues this cycle until it terminates, at which time it is removed from all queues.

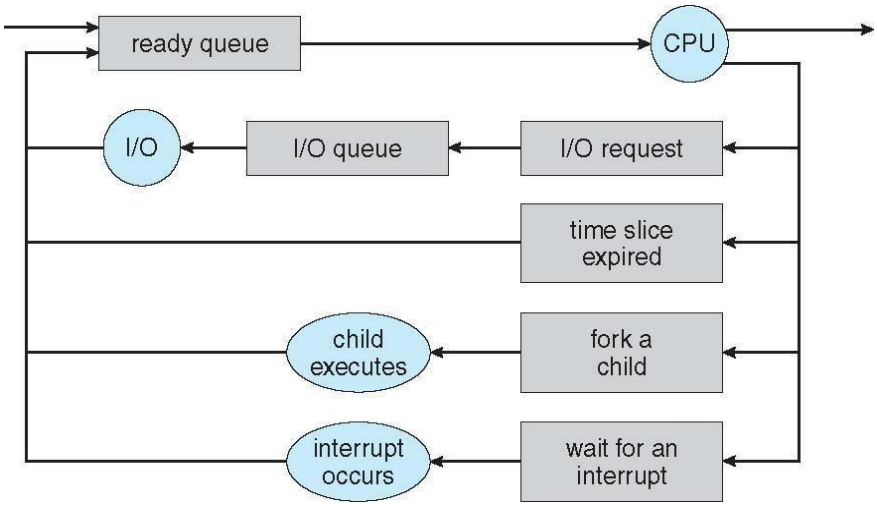


Figure: Queueing-diagram representation of process scheduling.

Schedulers

Schedulers are software which selects an available program to be assigned to CPU.

- A **long-term scheduler or Job scheduler** – selects jobs from the job pool (of secondary memory, disk) and loads them into the memory.
  If more processes are submitted, than that can be executed immediately, such processes will be in secondary memory. It runs infrequently, and can take time to select the next process.

- **The short-term scheduler, or CPU Scheduler** – selects job from memory and assigns the CPU to it. It must select the new process for CPU frequently.
- **The medium-term scheduler** - selects the process in ready queue and reintroduced into the memory.

Processes can be described as either:

- I/O-bound process – spends more time doing I/O than computations,
- CPU-bound process – spends more time doing computations and few I/O operations.

An efficient scheduling system will select a good mix of **CPU-bound** processes and **I/O bound** processes.

- If the scheduler selects **more I/O bound process**, then I/O queue will be full and ready queue will be empty.
- If the scheduler selects **more CPU bound process**, then ready queue will be full and I/O queue will be empty.

Time sharing systems employ a **medium-term scheduler**. It swaps out the process from ready queue and swap in the process to ready queue. When system loads get high, this scheduler will swap one or more processes out of the ready queue for a few seconds, in order to allow smaller faster jobs to finish up quickly and clear the system.

Advantages of medium-term scheduler –

- To remove process from memory and thus reduce the degree of multiprogramming (number of processes in memory).
- To make a proper mix of processes (CPU bound and I/O bound)

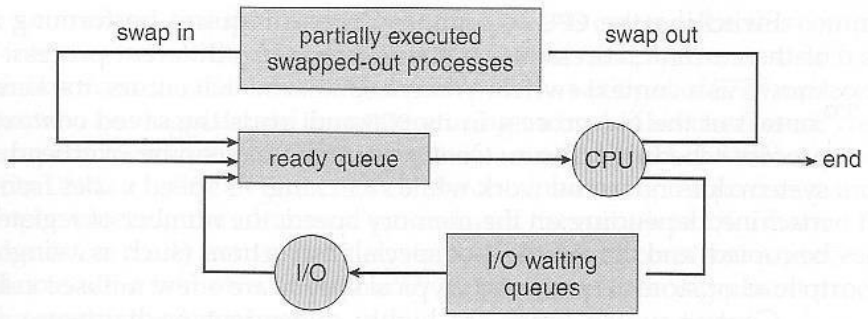


Figure 3.8 Addition of medium-term scheduling to the queueing diagram.

Context switching

- The task of switching a CPU from one process to another process is called context switching. Context-switch times are highly dependent on hardware support (Number of CPU registers).
- Whenever an interrupt occurs (hardware or software interrupt), the state of the currently running process is saved into the PCB and the state of another process is restored from the PCB to the CPU.
- Context switch time is an overhead, as the system does not do useful work while switching.

Operations on Processes

Q) Demonstrate the operations of process creation and process termination in UNIX

Process Creation

- A process may create several new processes. The creating process is called a **parent** process, and the new processes are called the **children** of that process. Each of these new processes may in turn create other processes. Every process has a unique process ID.
- On typical Solaris systems, the process at the top of the tree is the '**sched**' process with PID of 0. The **'sched'** process creates several children processes – **init**, **pageout** and **fsflush**. Pageout and fsflush are responsible for managing memory and file systems. The init process with a PID of 1, serves as a parent process for all user processes.

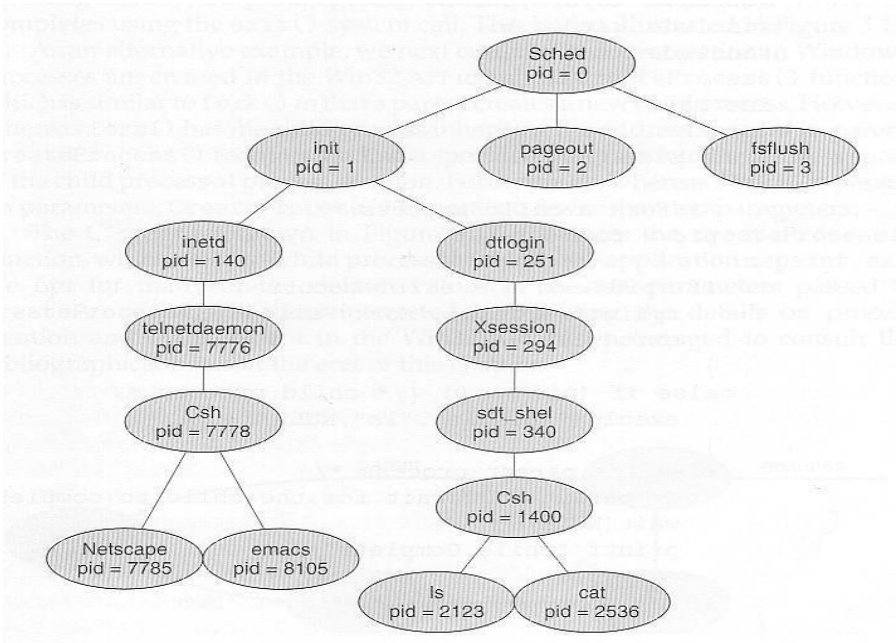


Figure 3.9 A tree of processes on a typical Solaris system.

A process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its task. When a process creates a subprocess, the subprocess may be able to obtain its resources in two ways:

- directly from the operating system
- Subprocess may take the resources of the parent process.

  The resource can be taken from parent in two ways –
  - The parent may have to partition its resources among its children
  - Share the resources among several children.

There are two options for the parent process after creating the child:

- Wait for the child process to terminate and then continue execution. The parent makes a wait() system call.
- Run concurrently with the child, continuing to execute without waiting.

Two possibilities for the address space of the child relative to the parent:

- The child may be an exact duplicate of the parent, sharing the same program and data segments in memory. Each will have their own PCB, including program counter, registers, and PID. This is the behaviour of the **fork** system call in UNIX.
- The child process may have a new program loaded into its address space, with all new code and data segments. This is the behaviour of the **spawn** system calls in Windows.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) {/* error occurred */
      fprintf(stderr, "Fork Failed");
      exit (-1) ;
    }
    else if (pid == 0} {/* child process */
      execlp("/bin/ls","ls",NULL);
    }
    else {/* parent process */
      /* parent will wait for the child to complete */
      wait(NULL);
      printf("Child Complete");
      exit (0) ;
    }
}
```

Figure 3.10 C program forking a separate process.

In UNIX OS, a child process can be created by **fork()** system call. The **fork** system call, if successful, returns the PID of the child process to its parents and returns a zero to the child process. If failure, it returns -1 to the parent. Process IDs of current process or its direct parent can be accessed using the getpid( ) and getppid( ) system calls respectively.

The parent waits for the child process to complete with the wait() system call. When the child process completes, the parent process resumes and completes its execution.

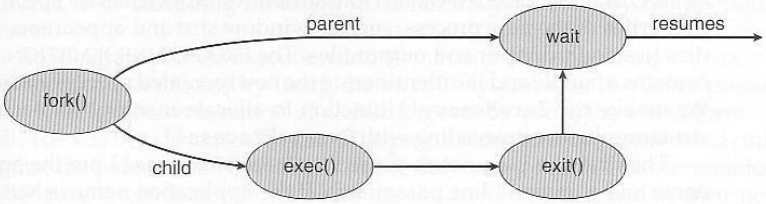


Figure 3.11 Process creation.

In windows the child process is created using the function **createprocess( )**. The createprocess( ) returns 1, if the child is created and returns 0, if the child is not created.

Process Termination

- A process terminates when it finishes executing its last statement and asks the operating system to delete it, by using the **exit** () system call. All of the resources assigned to the process like memory, open files, and I/O buffers, are deallocated by the operating system.
- A process can cause the termination of another process by using appropriate system call. The parent process can terminate its child processes by knowing of the PID of the child.
- A parent may terminate the execution of children for a variety of reasons, such as:
    - The child has exceeded its usage of the resources, it has been allocated.
    - The task assigned to the child is no longer required.
    - The parent is exiting, and the operating system terminates all the children. This is called **cascading termination**.

Interprocess Communication

Q) What is interprocess communication? Explain types of IPC.

Interprocess Communication- Processes executing may be either co-operative or independent processes.

- ***Independent Processes*** – processes that cannot affect other processes or be affected by other processes executing in the system.
- ***Cooperating Processes*** – processes that can affect other processes or be affected by other processes executing in the system.

Co-operation among processes are allowed for following reasons –

- **Information Sharing** - There may be several processes which need to access the same file. So the information must be accessible at the same time to all users.
- **Computation speedup** - Often a solution to a problem can be solved faster if the problem can be broken down into sub-tasks, which are solved simultaneously (particularly when multiple

processors are involved.)

- **Modularity** - A system can be divided into cooperating modules and executed by sending information among one another.
- **Convenience** - Even a single user can work on multiple tasks by information sharing.

Cooperating processes require some type of inter-process communication. This is allowed by two models:

1. Shared Memory systems
2. Message passing systems.

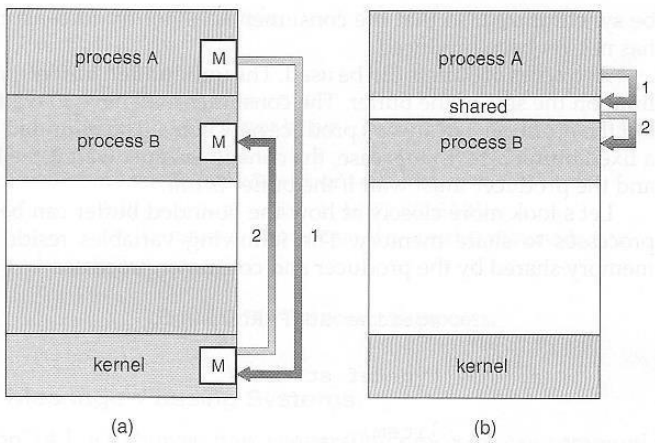


Figure 3.13 Communications models. (a) Message passing. (b) Shared memory.

Sl No	**Shared Memory**	**Message passing**
1.	A region of memory is shared by communicating processes, into which the information is written and read	Message exchange is done among the processes by using objects.
2.	Useful for sending large block of data	Useful for sending small data.
3.	System call is used only to create shared memory	System call is used during every read and write operation.
4.	Message is sent faster, as there are no system calls	Message is communicated slowly.

- **Shared Memory** is faster once it is set up, because no system calls are required and access occurs at normal memory speeds. Shared memory is generally preferable when large amounts of information must be shared quickly on the same computer.
- **Message Passing** requires system calls for every message transfer, and is therefore slower, but it is simpler to set up and works well across multiple computers. Message passing is generally preferable when the amount and/or frequency of data transfers is small.

Shared-Memory Systems

- A region of shared-memory is created within the address space of a process, which needs to communicate. Other process that needs to communicate uses this shared memory.
- The form of data and position of creating shared memory area is decided by the process. Generally, a few messages must be passed back and forth between the cooperating processes first in order to set up and coordinate the shared memory access.
- The process should take care that the two processes will not write the data to the shared memory at the same time.

Producer-Consumer Example Using Shared Memory

- This is a classic example, in which one process is producing data and another process is consuming the data.
- The data is passed via an intermediary buffer (shared memory). The producer puts the data to the buffer and the consumer takes out the data from the buffer. A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced. In this situation, the consumer must wait until an item is produced.
- There are two types of buffers into which information can be put –
    - Unbounded buffer
    - Bounded buffer

- **With Unbounded buffer**, there is no limit on the size of the buffer, and so on the data produced by producer. But the consumer may have to wait for new items.

- **With bounded-buffer** – As the buffer size is fixed. The producer has to wait if the buffer is full and the consumer has to wait if the buffer is empty.

This example uses shared memory as a circular queue. The **in** and **out** are two pointers to the array. Note in the code below that only the producer changes "in", and only the consumer changes "out".

- First the following data is set up in the shared memory area:

```
#define BUFFER_SIZE 10

typedef struct {
    . . .
}item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- The producer process –
  Note that the buffer is full when [(in+1) % BUFFER_SIZE == out]

```
item nextProduced;

while (true) {
     /* produce an item in nextProduced */
     while (((in + 1) % BUFFER-SIZE) == out)
        ; /* do nothing */
     buffer[in] = nextProduced;
     in = (in + 1) % BUFFER_SIZE;
}
```

Figure The producer process.

- The consumer process –
  Note that the buffer is empty when [in == out]

```
item nextConsumed;

while (true) {
     while (in == out)
        ; //do nothing

     nextConsumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     /* consume the item in nextConsumed */
}
```

Figure The consumer process.

Message-Passing Systems

A mechanism to allow process communication without sharing address space. It is used in distributed systems.

- Message passing systems uses system calls for "send message" and "receive message".
- A communication link must be established between the cooperating processes before messages can be sent.
- There are three methods of creating the link between the sender and the receiver-
  - Direct or indirect communication (naming)
  - Synchronous or asynchronous communication (Synchronization)
  - Automatic or explicit buffering.

1. Naming

Processes that want to communicate must have a way to refer to each other. They can use either direct or indirect communication.

a) Direct communication the sender and receiver must explicitly know each other's name. The syntax for send() and receive() functions are as follows-

- **send** (*P*, *message*) – send a message to process P
- **receive**(*Q*, *message*) – receive a message from process Q

Properties of communication link:

- A link is established automatically between every pair of processes that wants to communicate. The processes need to know only each other's identity to communicate.
- A link is associated with exactly one pair of communicating processes
- Between each pair, there exists exactly one link.

Types of addressing in direct communication –

- Symmetric addressing – the above-described communication is symmetric communication. Here both the sender and the receiver processes have to name each other to communicate.
- Asymmetric addressing – Here only the sender's name is mentioned, but the receiving data can be from any system.

  send (P, message) --- Send a message to process ***P***

  receive (id, message). Receive a message from any process

Disadvantages of direct communication – any changes in the identifier of a process, may have to change the identifier in the whole system (sender and receiver), where the messages are sent and received.

b) Indirect communication uses shared mailboxes, or ports.

A mailbox or port is used to send and receive messages. Mailbox is an object into which messages can be sent and received. It has a unique ID. Using this identifier messages are sent and received.

Two processes can communicate only if they have a shared mailbox. The send and receive functions are –

- **send** (*A*, *message*) – send a message to mailbox A
- **receive** (*A*, *message*) – receive a message from mailbox A

Properties of communication link:

- A link is established between a pair of processes only if they have a shared mailbox

- A link may be associated with more than two processes
- Between each pair of communicating processes, there may be any number of links, each link is associated with one mailbox.
- A mail box can be owned by the operating system. It must take steps to –
  - create a new mailbox
  - send and receive messages from mailbox
  - delete mailboxes.

2. Synchronization

The send and receive messages can be implemented as either **blocking** or **non-blocking**.

- **Blocking (synchronous) send -** sending process is blocked (waits) until the message is received by receiving process or the mailbox.
- **Non-blocking (asynchronous) send -** sends the message and continues (does not wait)

- **Blocking (synchronous) receive -** The receiving process is blocked until a message is available
- **Non-blocking (asynchronous) receive -** receives the message without block. The received message may be a valid message or null.

3. Buffering

When messages are passed, a temporary queue is created. Such queue can be of three capacities:

- **Zero capacity** – The buffer size is zero (buffer does not exist). Messages are not stored in the queue. The senders must block until receivers accept the messages.
- **Bounded capacity-** The queue is of fixed size(n). Senders must block if the queue is full. After sending 'n' bytes the sender is blocked.
- Unbounded capacity **- The queue is of infinite capacity. The sender never block**

MODULE 2

MULTITHREADED PROGRAMMING

- A thread is a basic unit of CPUutilization.
- It consistsof
    - thread ID
    - PC
    - register-set and
    - stack.
- It shares with other threads belonging to the same process its code-section &data-section.
- A traditional (or heavy weight) process has a single thread ofcontrol.
- If a process has multiple threads of control, it can perform more than one task at a time.

such a process is called **multithreaded process**

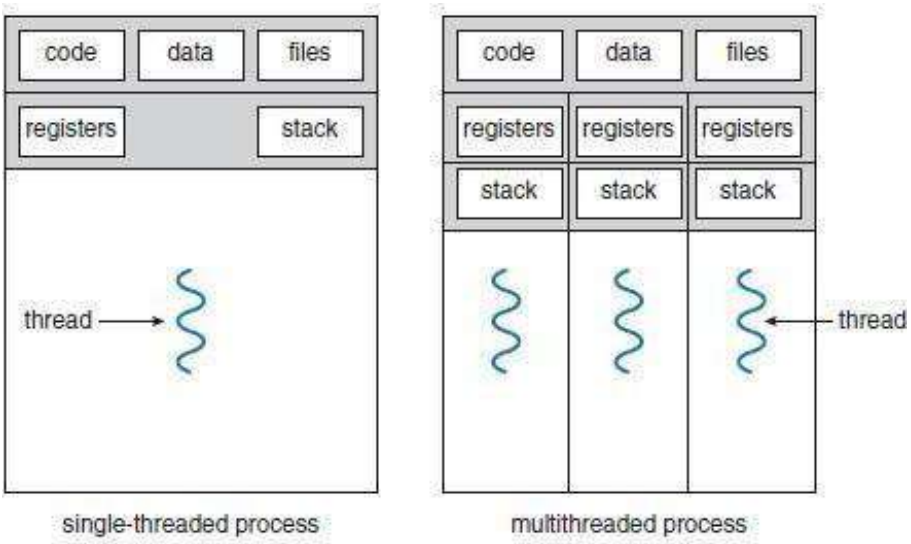


Fig: Single-threaded and multithreaded processes

Motivation for Multithreaded Programming

1. The software-packages that run on modern PCs aremultithreaded.An application is implemented as a separate process with several threads of control. For ex: A word processor mayhave
    - first thread for displaying graphics
    - second thread for responding to keystrokesand
    - Thirdthread for performing grammarchecking.

2. In some situations, a single application may be required to perform several similartasks. For ex: A web-server may create a separate thread for each client requests. This allows the server to service several concurrent requests.

3. RPC servers aremultithreaded.
   - When a server receives a message, it services the message using separate concurrent threads.

4. Most OS kernels aremultithreaded;
   - Several threads operate in kernel, and each thread performs a specific task, suchasmanaging devices or interrupt handling.

Benefits of Multithreaded Programming

- ***Responsiveness*** A program may be allowed to continue running even if part of it is blocked. Thus, increasing responsiveness to the user.

- ***Resource Sharing*** By default, threads share the memory (and resources) of the process to which they belong. Thus, an application is allowed to have several different threads of activity within the sameaddress-space.
- ***Economy*** Allocating memory and resources for process-creation is costly. Thus, it is more economical to create and context-switchthreads.
- ***Utilization of Multiprocessor Architectures*** In a multiprocessor architecture, threads may be running in parallel on different processors. Thus, parallelism will beincreased.

MULTITHREADING MODELS

- Support for threads may be provided ateither
  1. The user level, for **user threads** or
  2. By the kernel, for **kernel threads**.
- User-threads are supported above the kernel and are managed withoutkernelsupport. Kernel-threads are supported and managed directly by the OS.
- Three ways of establishing relationship between user-threads &kernel-threads:
  1. Many-to-onemodel
  2. One-to-one modeland
  3. Many-to-manymodel.

Many-to-One Model

- Many user-level threads are mapped to one kernel thread.

  Advantages:

  - Thread management is done by the thread library in user space, so it isefficient.

  Disadvantages:

  - The entire process will block if a thread makes a blockingsystem-call.
  - Multiple threads are unable to run in parallel onmultiprocessors.
- Forexample:
  - Solaris green threads
  - GNU portable threads.

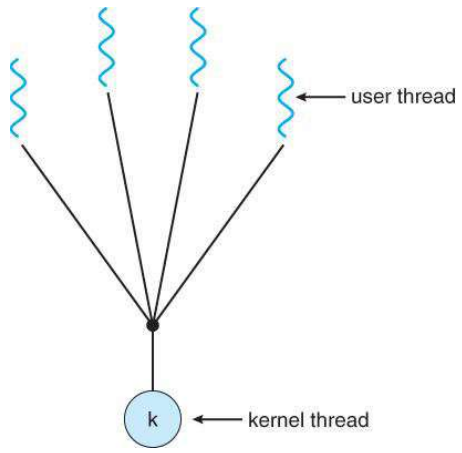


Fig: Many-to-one model

One-to-One Model

- Each user thread is mapped to a kernel thread.

  Advantages:

  - It provides more concurrency by allowing another thread to run when a thread makes a blockingsystem-call.
  - Multiple threads can run in parallel on multiprocessors.

  Disadvantage:

  - Creating a user thread requires creating the corresponding kernel thread.
- For example:
  - Windows NT/XP/2000, Linux

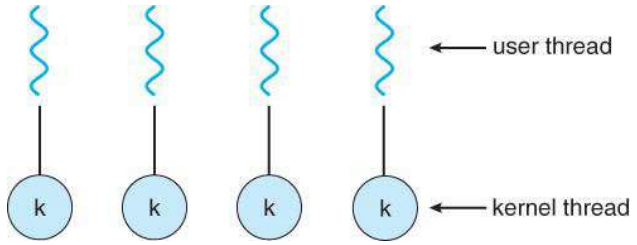


Fig: one-to-one model

Many-to-Many Model

- Many user-level threads are multiplexed to a smaller number of kernel threads.

  Advantages:

  - Developers can create as many user threads as necessary
  - The kernel threads can run in parallel on amultiprocessor.
  - When a thread performs a blocking system-call, kernel can schedule another thread for execution.

Two Level Model

- A variation on the many-to-many model is the two level-model
- Similar to M:N, except that it allows a user thread to be bound to kernelthread.
- forexample:
  - HP-UX
  - Tru64 UNIX

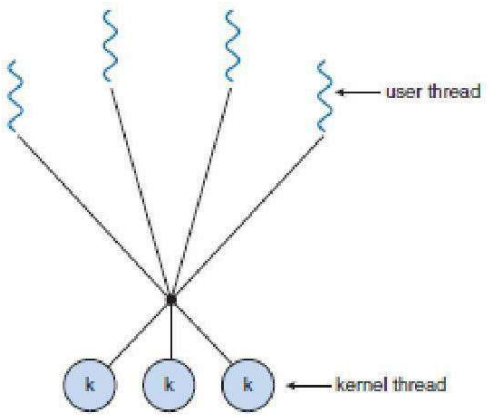


Fig: Many-to-many model

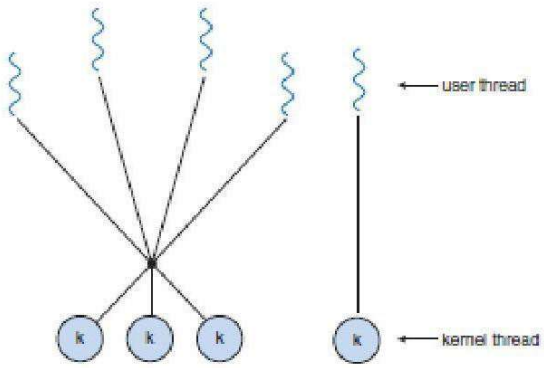


Fig: Two-level model

Thread Libraries

- It provides the programmer with an API for the creation and management ofthreads.

- Two ways of implementation:
  1. ***First Approach:***
     Provides a library entirely in user space with no kernel support. All code and data structures for the library exist in the user space.
  2. ***SecondApproach***
     Provides a library entirely in user space with no kernel support. All code and data structures for the library exist in the user space.

Three main threadlibraries:

1. POSIXP threads
2. Win32 and
3. Java.

Pthreads

- This is a POSIX standard API for thread creation andsynchronization.
- This is a specification for thread-behavior, not an implementation.
- OS designers may implement the specification in any way theywish.
- Commonly used in: UNIX andSolaris.

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* the thread */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   if (argc != 2) {
      fprintf(stderr,"usage: a.out <integer value>\n");
      return -1;
   }
   if (atoi(argv[1]) < 0) {
      fprintf(stderr,"%d must be >= 0\n",atoi(argv[1]));
      return -1;
   }

   /* get the default attributes */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid,&attr,runner,argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}
```

```
/* The thread will begin control in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```

Win32 threads

- Implements the one-to-onemapping
- Each threadcontains
  - A threadid
  - Registerset
  - Separate user and kernelstacks
  - Private data storagearea
- The register set, stacks, and private storage area are known as the context of the threads The primary data structures of a thread include:
  - ETHREAD (executive threadblock)
  - KTHREAD (kernel threadblock)
  - TEB (thread environmentblock)

Java Threads

- Threads are the basic model of program-executionin
  - Java program and
  - Java language.
- The API provides a rich set of features for the creation and management of threads.
- All Java programs comprise at least a single thread ofcontrol.
- Two techniques for creating threads:
  1. Create a new class that is derived from the Thread class and override its run() method.
  2. Define a class that implements the Runnable interface. The Runnable interface is defined as follows:

```
public interface Runnable
{
   public abstract void run();
}
```

THREADING ISSUES

fork() and exec() System-calls

- fork() is used to create a separate, duplicateprocess.
- If one thread in a program calls fork(),then
    1. Some systems duplicates all threads and
    2. Other systems duplicate only the thread that invoked the forkO.
- If a thread invokes the exec(), the program specified in the parameter to exec() will replace the entire process including allthreads.

Thread Cancellation

- This is the task of terminating a thread before it hascompleted.
- Target thread is the thread that is to be cancelled
- Thread cancellation occurs in two differentcases:
    1. ***Asynchronous cancellation***: One thread immediately terminates the targetthread.
    2. ***Deferred cancellation***: The target thread periodically checks whether it should be terminated.

Signal Handling

- In UNIX, a signal is used to notify a process that a particular event hasoccurred.
- All signals follow thispattern:
    1. A signal is generated by the occurrence of a certainevent.
    2. A generated signal is delivered to aprocess.
    3. Once delivered, the signal must behandled.
- A signal handler is used to processsignals.
- A signal may be received either synchronously or asynchronously, depending on thesource.
    1. ***Synchronoussignals***
        - Delivered to the same process that performed the operation causing the signal.
        - E.g. illegal memory access and division by 0.
    2. ***Asynchronoussignals***
        - Generated by an event external to a running process.
        - E.g. user terminating a process with specific keystrokes<ctrl><c>.

- Every signal can be handled by one of two possiblehandlers:

    1. A Default SignalHandler

    - Run by the kernel when handling the signal.

    2. A User-defined SignalHandler

    - Overrides the default signal handler.

- In ***single-threaded programs***, delivering signals is simple (since signals are always delivered to a process).
- In ***multithreaded programs***, delivering signals is more complex. Then, the following options exist:
    1. Deliver the signal to the thread to which the signal applies.
    2. Deliver the signal to every thread in process
    3. Deliver the signal to certain threads in the process.
    4. Assign a specific thread to receive all signals for the process.

THREAD POOLS

- The basic idea is to
    - create a no. of threads at process-startup and
    - place the threads into a pool (where they sit and wait for work).
- Procedure:
    1. When a server receives a request, it awakens a thread from the pool.
    2. If any thread is available, the request is passed to it for service.
    3. Once the service is completed, the thread returns to the pool.
- Advantages:
    - Servicing a request with an existing thread is usually faster than waiting to create a thread.
    - The pool limits the no. of threads that exist at any one point.
- No. of threads in the pool can be based on actors such as
    - no. of CPUs
    - amount of memory and
    - expected no. of concurrent client-requests.

THREAD SPECIFIC DATA

- Threads belonging to a process share the data of the process.
- this sharing of data provides one of the benefits of multithreadedprogramming.
- In some circumstances, each thread might need its own copy of certain data. We will call such data **thread-specific data**.
- For example, in a transaction-processing system, we might service each transaction in a separatethread.

- Furthermore, each transaction may be assigned a unique identifier. To associate each thread with its unique identifier, we could use thread-specificdata.

SCHEDULER ACTIVATIONS

- Both M:M and Two-level models require communication to maintain the appropriate number of kernel threads allocated to theapplication.
- Scheduler activations provide **upcalls**a communication mechanism from the kernel to the threadlibrary
- This communication allows an application to maintain the correct number kernel threads
- One scheme for communication between the user-thread library and the kernel is known as **scheduler activation.**

PROCESS SCHEDULING

Basic Concepts

- In a single-processor system,
  - Only one process may run at a time.
  - Other processes must wait until the CPU is rescheduled.
- Objective ofmultiprogramming:
  - To have some process running at all times, in order to maximize CPU utilization.

CPU-I/0 Burst Cycle

- Process execution consists of a cycleof
  - CPU execution and
  - I/O wait
- Process execution begins with a CPU burst, followed by an I/O burst, then another CPU burst, etc…
- Finally, a CPU burst ends with a request to terminateexecution.
- An I/O-bound program typically has many short CPUbursts.
- A CPU-bound program might have a few long CPU bursts.

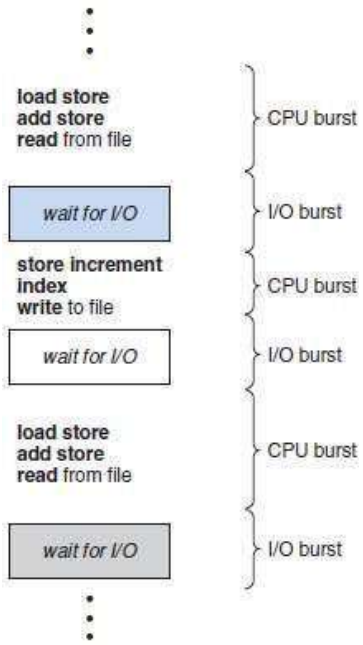


Fig Alternating sequence of CPU and I/O bursts

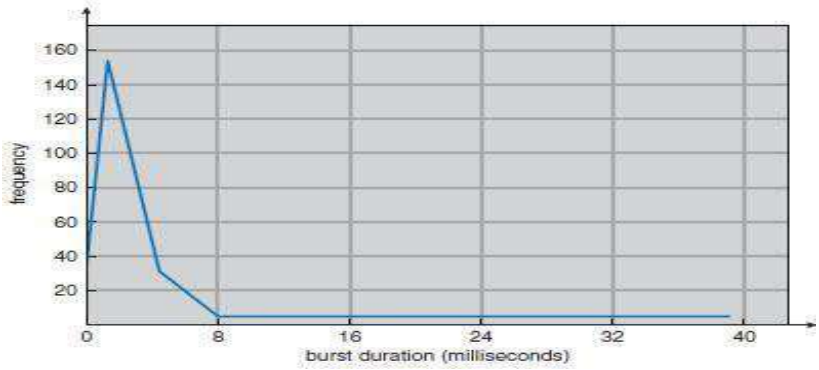


Fig: Histogram of CPU-burst durations

CPU Scheduler

- Thisscheduler
  - selects a waiting-process from the ready-queue and
  - allocates CPU to the waiting-process.
- The ready-queue could be a FIFO, priority queue, tree andlist.
- The records in the queues are generally process control blocks (PCBs) of theprocesses.

CPU Scheduling

- Four situations under which CPU scheduling decisions takeplace:
  1. When a process switches from the running state to the waiting state. For ex; I/O request.
  2. When a process switches from the running state to the ready state. For ex: when an interrupt occurs.
  3. When a process switches from the waiting state to the ready state. For ex: completion of I/O.
  4. When a process terminates.
- Scheduling under 1 and 4 is non- preemptive. Scheduling under 2 and 3 is preemptive.

Non Preemptive Scheduling

- Once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either
  - by terminating or
  - by switching to the waiting state.

Preemptive Scheduling

- This is driven by the idea of prioritizedcomputation.
- Processes that are runnable may be temporarilysuspended
- Disadvantages:

Dispatcher

- It gives control of the CPU to the process selected by the short-termscheduler.
- The functioninvolves:
    1. Switchingcontext
    2. Switching to user mode&
    3. Jumping to the proper location in the user program to restart that program
- It should be as fast as possible, since it is invoked during every process switch.
- ***Dispatch latency*** means the time taken by the dispatcherto
    - stop one process and
    - start another running.

SCHEDULING CRITERIA:

In choosing which algorithm to use in a particular situation, depends upon the properties of the various algorithms.Many criteria have been suggested for comparing CPU-scheduling algorithms. The criteria include the following:

1. **CPU utilization:** We want to keep the CPU as busy as possible. Conceptually, CPU utilization can range from 0 to 100 percent. In a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily used system).
2. **Throughput:** If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called throughput. For long processes, this rate may be one process per hour; for short transactions, it may be ten processes per second.

3. **Turnaround time.** This is the important criterion which tells how long it takes to execute that process. The interval from the time of submission of a process to the time of completion is the turnaround time. Turnaround time is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/0.
4. **Waiting time:** The CPU-scheduling algorithm does not affect the amount of time during which a process executes or does I/0, it affects only the amount of time that a process spends waiting in the ready queue.Waiting time is the sum of the periods spent waiting in the ready queue.
5. **Response time:**In an interactive system, turnaround time may not be the best criterion. Often, a process can produce some output fairly early and can continue computing new results while previous results are being output to the user. Thus, another measure is the time from the submission of a request until the first response is produced. This measure, called response time, is the time it takes to start responding, not the time it takes to output the response. The turnaround time is

SCHEDULING ALGORITHMS

- CPU scheduling deals with the problem of deciding which of the processes in the ready-queue is to be allocated theCPU.
- Following are some schedulingalgorithms:
  1. FCFS scheduling (First Come FirstServed)
  2. Round Robin scheduling
  3. SJF scheduling (Shortest JobFirst)
  4. SRT scheduling
  5. Priority scheduling
  6. Multilevel Queue schedulingand
  7. Multilevel Feedback Queuescheduling

FCFS Scheduling

- The process that requests the CPU first is allocated the CPUfirst.
- The implementation is easily done using a FIFOqueue.
- Procedure:
  1. When a process enters the ready-queue, its PCB is linked onto the tail of thequeue.
  2. When the CPU is free, the CPU is allocated to the process at the queue'shead.
  3. The running process is then removed from the queue.

- Advantage:
  1. Code is simple to write & understand.
- Disadvantages:
  1. **Convoy effect:** All other processes wait for one big process to get off theCPU.
  2. Non-preemptive (a process keeps the CPU until it releasesit).
  3. Not good for time-sharingsystems.
  4. The average waiting time is generally notminimal.

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Example: Suppose that the processes arrive in the order P1, P2,P3.

- The Gantt Chart for the schedule is asfollows:

- Waiting time for *P1* = 0; *P2* = 24; *P3* =27

  Average waiting time: (0 + 24 + 27)/3 = 17ms

- The Gantt chart for the schedule is asfollows:

- Waiting time for P1 = 6;P2 = 0; P3 =3
- Average waiting time: (6 + 0 + 3)/3 = 3ms

SJF Scheduling

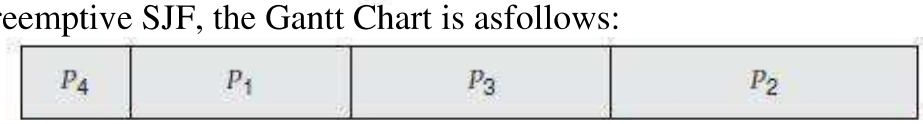
- The CPU is assigned to the process that has the smallest next CPUburst.
- If two processes have the same length CPU burst, FCFS scheduling is used to break thetie.
- For long-term scheduling in a batch system, we can use the process time limit specified by the user, as the'length'
- SJF can't be implemented at the level of short-term scheduling, because there is no way to know the length of the next CPUburst
- Advantage:
  1. The SJF is optimal, i.e. it gives the minimum average waiting time for a given set of processes.
- Disadvantage:
  1. Determining the length of the next CPU burst.

- SJF algorithm may be either 1) non-preemptive or 2)preemptive.
  1. ***Non preemptiveSJF***
     The current process is allowed to finish its CPU burst.
  2. ***PreemptiveSJF***
     If the new process has a shorter next CPU burst than what is left of the executing process, that process is preempted. It is also known as **SRTF** scheduling (Shortest-Remaining-Time-First).

- Example (for non-preemptive SJF): Consider the following set of processes, with the length of the CPU-burst time given inmilliseconds.

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

- For non-preemptive SJF, the Gantt Chart is asfollows:



- Waiting time for P1 = 3; P2 = 16; P3 = 9; 4=0 Average waiting time: (3 + 16 + 9 + 0)/4= 7

preemptive SJF/SRTF: Consider the following set of processes, with the length

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

of the CPU- burst time given inmilliseconds.

- For preemptive SJF, the Gantt Chart is asfollows:

- The average waiting time is ((10 - 1) + (1 - 1) + (17 - 2) + (5 - 3))/4 = 26/4 =6.5.

Priority Scheduling

- A priority is associated with eachprocess.
- The CPU is allocated to the process with the highestpriority.
- Equal-priority processes are scheduled in FCFSorder.
- Priorities can be defined either internally orexternally.
    1. ***Internally-defined*** priorities.
        - Use some measurable quantity to compute the priority of a process.
        - For example: time limits, memory requirements, no. f open files.
    2. ***Externally-defined*** priorities.
        - Set by criteria that are external to the OS For example:
        - importance of the process, political factors
- Priority scheduling can be either preemptive or non-preemptive.
    1. ***Preemptive***

       The CPU is preempted if the priority of the newly arrived process is higher than the priority of the currently running process.
    2. ***Non Preemptive***

       The new process is put at the head of the ready-queue

- Advantage:
    - Higher priority processes can be executed first.
- Disadvantage:
    - Indefinite blocking, where low-priority processes are left waiting

• Example: Consider the following set of processes, assumed to have arrived at time 0, in the order PI, P2, ..., P5, with the length of the CPU-burst time given inmilliseconds.

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

• The Gantt chart for the schedule is asfollows:

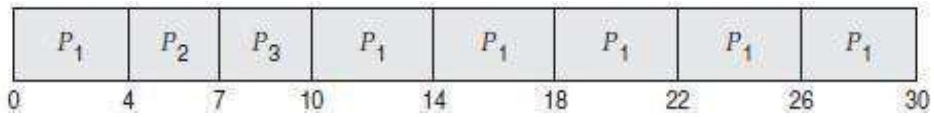
• The average waiting time is 8.2milliseconds.

Round Robin Scheduling

• Designed especially for timesharingsystems.
• It is similar to FCFS scheduling, but with preemption.
• A small unit of time is called a ***time quantum****(or timeslice).*
• Time quantum is ranges from 10 to 100ms.
• The ready-queue is treated as a **circularqueue**.
• The CPUscheduler
  - goes around the ready-queue and
  - allocates the CPU to each process for a time interval of up to 1 time quantum.
• To implement:
  The ready-queue is kept as a FIFO queue of processes
• CPUscheduler
  1. Picks the first process from theready-queue.
  2. Sets a timer to interrupt after 1 time quantumand
  3. Dispatches theprocess.
• One of two things will thenhappen.
  1. The process may have a CPU burst of less than 1 time quantum. In this case, the process itself will release the CPU voluntarily.
  2. If the CPU burst of the currently running process is longer than 1 time quantum, the timer will go off and will cause an interrupt to the OS. The process will be put at the tail of the ready-queue.
• Advantage:
  - Higher average turnaround than SJF.
• Disadvantage:
  - Better response time than SJF.

Process	Burst Time
P_1	24
P_2	3
P_3	3

- The Gantt chart for the schedule is asfollows:



- The average waiting time is 17/3 = 5.66milliseconds.

- *The RR scheduling algorithm is preemptive.*

  No process is allocated the CPU for more than *1* time quantum in a row. If a process' CPU burst exceeds *1* time quantum, that process is preempted and is put back in the ready- queue.

- The performance of algorithm depends heavily on the size of the time quantum.
  1. If time quantum=very large, RR policy is the same as the FCFSpolicy.
  2. If time quantum=very small, RR approach appears to the users as though each of n processes has its own processor running at l/n the speed of the real processor.

- In software, we need to consider the effect of context switching on the performance of RR scheduling
  1. Larger the time quantum for a specific process time, less time is spend on context switching.
  2. The smaller the time quantum, more overhead is added for the purpose of context- switching.

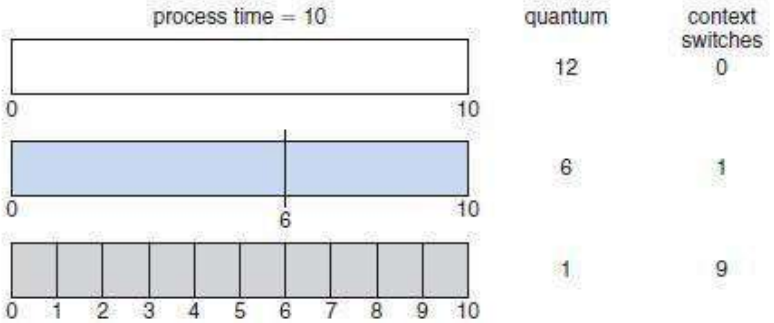
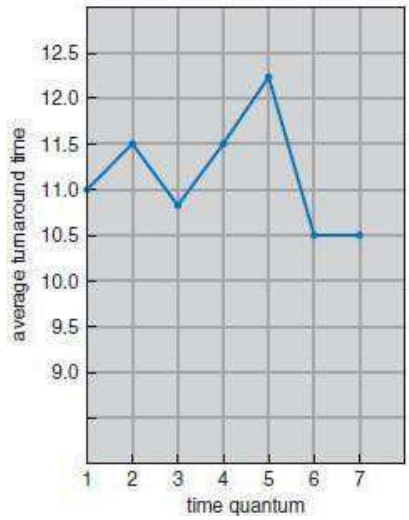


Fig: How a smaller time quantum increases context switches



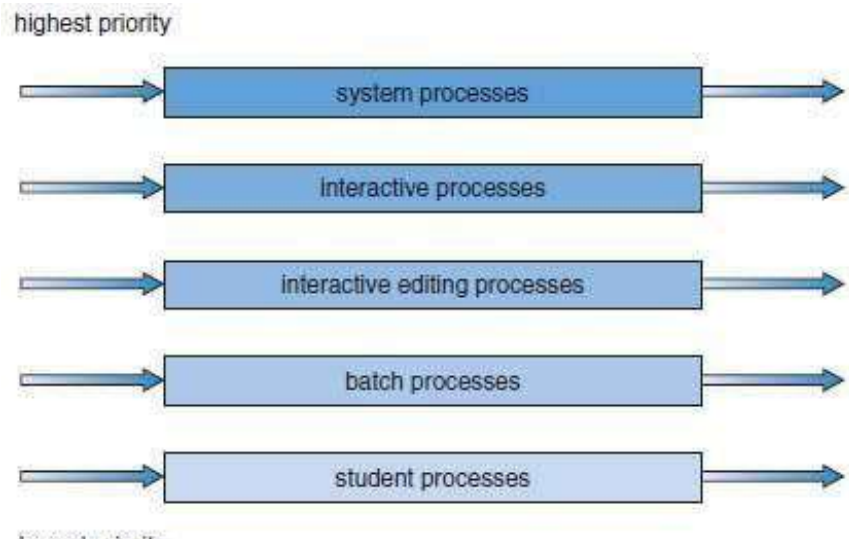
process	time
P_1	6
P_2	3
P_3	1
P_4	7

Fig: How turnaround time varies with the time quantum

Multilevel Queue Scheduling

- Useful for situations in which processes are easily classified into different groups.
- For example, a common division is made between
  - foreground (or interactive) processes and
  - background (or batch) processes.
- The ready-queue is partitioned into several separate queues (Figure2.19).
- The processes are permanently assigned to one queue based on some property like
  - memory size
  - process priority or
  - process type.
- Each queue has its own scheduling algorithm.

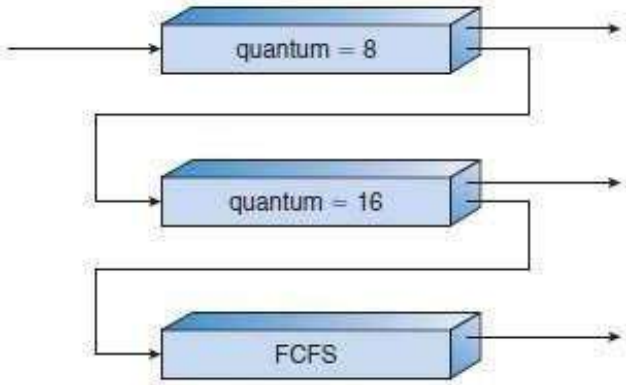
  For example, separate queues might be used for foreground and background processes.



- There must be scheduling among the queues, which is commonly implemented as fixed-priority preemptive scheduling.
- For example, the foreground queue may have absolute priority over the background queue.
- **Time slice**: each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR 20% to background in FCFS

Multilevel Feedback Queue Scheduling

- A process may move between queues
- The basic idea: Separate processes according to the features of their CPU bursts. Forexample
  1. If a process uses too much CPU time, it will be moved to a lower-priority queue. This scheme leaves I/O-bound and interactive processes in the higher-priority queues.
  2. If a process waits too long in a lower-priority queue, it may be moved to a higher-priority queue This form of aging prevents starvation.



.

Figure 2.20 Multilevel feedback queues

In general, a multilevel feedback queue scheduler is defined by the followingparameters:

1. The number ofqueues.
2. The scheduling algorithm for eachqueue.
3. The method used to determine when to upgrade a process to a higher priorityqueue.
4. The method used to determine when to demote a process to a lower priorityqueue.
5. The method used to determine which queue a process will enter when that process needs service

MULTIPLE PROCESSOR SCHEDULING

- If multiple CPUs are available, the scheduling problem becomes morecomplex.
- Twoapproaches:

AsymmetricMultiprocessing

The basic idea is:

- A master server is a single processor responsible for all scheduling decisions, I/O processing and other systemactivities.
- The other processors execute only user code.
- Advantage: This is simple because only one processor accesses the system data structures, reducing the need for data sharing.

Symmetric Multiprocessing

The basic idea is:

- Each processor is self-scheduling.
- To do scheduling, the scheduler for eachprocessor
- Examines the ready-queue and
- Selects a process to execute.

Restriction: We must ensure that two processors do not choose the same process and that processes are not lost from the queue.

Processor Affinity

- In SMP systems,
  1. Migration of processes from one processor to another are avoided and
  2. Instead processes are kept running on same processor. This is known as processor affinity.
- Two forms:
  1. ***SoftAffinity***
     - When an OS try to keep a process on one processor because of policy, but cannot guarantee it will happen.
     - It is possible for a process to migrate between processors.
  2. ***Hard Affinity***
     - When an OS have the ability to allow a process to specify that it is not to migrate to other processors. Eg: Solaris OS

Load Balancing

- This attempts to keep the workload evenly distributed across all processors in an SMPsystem.
- Twoapproaches:
  1. ***PushMigration***

     A specific task periodically checks the load on each processor and if it finds an imbalance, it evenly distributes the load to idle processors.
  2. ***PullMigration***

Symmetric Multithreading

- The basic idea:
    1. Create multiple logical processors on the same physical processor.
    2. Present a view of several logical processors to the OS.
- Each logical processor has its own architecture state, which includes general-purpose and machine-state registers.
- Each logical processor is responsible for its own interrupt handling.
- SMT is a feature provided in hardware, notsoftware.

THREAD SCHEDULING

- On OSs, it is kernel-level threads but not processes that are being scheduled by theOS.
- User-level threads are managed by a thread library, and the kernel is unaware ofthem.
- To run on a CPU, user-level threads must be mapped to an associated kernel-levelthread.

Contention Scope

- Twoapproaches:
    1. ***Process-Contention scope***
        - On systems implementing the many-to-one and many-to-many models, the thread library schedules user-level threads to run on an available LWP.
        - Competition for the CPU takes place among threads belonging to the sameprocess.
    2. ***System-Contentionscope***
        - The process of deciding which kernel thread to schedule on theCPU.
        - Competition for the CPU takes place among all threads in thesystem.
        - Systems using the one-to-one model schedule threads using onlySCS.

Pthread Scheduling

- Pthread API that allows specifying either PCS or SCS during threadcreation.
- Pthreads identifies the following contention scopevalues:
    1. PTHREAD_SCOPEJPROCESS schedules threads using PCSscheduling.
    2. PTHREAD-SCOPE_SYSTEM schedules threads using SCSscheduling.
- Pthread IPC provides following two functions for getting and setting the contention scopepolicy:
    1. pthread_attr_setscope(pthread_attr_t *attr, intscope)
    2. pthread_attr_getscope(pthread_attr_t *attr, int*scop)